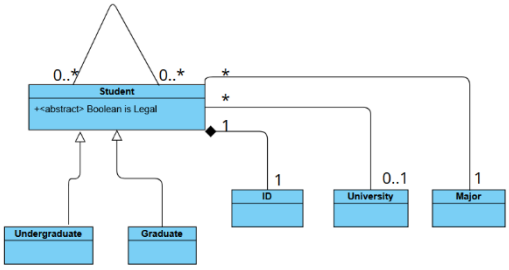
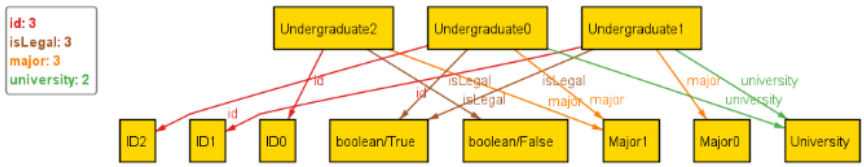
Formal Modeling

1.a)



1.b)

```
1   open util/boolean
2
3   sig University {}
4   sig Major {}
5   sig ID {}
6
7   abstract sig Student {
8       id: one ID,
9       major: one Major,
10      university: lone University,
11      isLegal: one Bool,
12      classmates: set Student
13  }
14  sig Undergraduate extends Student {}
15  sig Graduate extends Student {}
16  fact SystemConstraints {
17      // Every student has a unique student ID, and no unassigned IDs
18      all i: ID | one id.i
19      // Registered students are legal, unregistered are not
20      all s: Student | s.isLegal = True iff some s.university
21      // Classmate constraints
22      all s1, s2: Student | s2 in s1.classmates iff {
23          s1 != s2 // A student cannot be his/her own classmate
24          some s1.university
25          s1.university = s2.university // Must be registered at same university
26          s1.major = s2.major           // Must have same major
27          // Graduates and undergraduates are never classmates
28          (s1 in Undergraduate and s2 in Undergraduate) or
29          (s1 in Graduate and s2 in Graduate)
30      }
31  }
32  pred show {}
    Execute
33  run show for 2 University, 3 Major, 3 Student, 3 ID
```



2.a)

UZHBus musst be empty so there is no bus. In line 14 there is a contradiction in this example. As b1 and b2 are the same bus and as it checks if the stations are not the same it can never evaluate to true.

For the Stationes all need to be one. As there are 4 stationes but in the run command they are restricted to one and at the same time they need to be all reachable and have a next station.

2.b)

```
1
2   abstract sig UZHBusStation {
3       next: one UZHBusStation
4   }
5   // (i) There are exactly 4 UZHBusStations: ZurichCity, Irchel, Oerlikon, and Schlieren
6   one sig ZurichCity, Irchel, Oerlikon, Schlieren extends UZHBusStation {}
7
8   sig UZHBus {
9       station: lone UZHBusStation
10  }
11
12  fact BusConstraints {
13      // (ii) The UZHBusStations form one directed circle.
14      all s: UZHBusStation | UZHBusStation in s.^next and some s.next
15
16      // (iii) There is no UZHBusStation between Oerlikon and Schlieren.
17      Oerlikon.next = Schlieren
18
19      // (iv) There can be at most one UZHBus at each UZHBusStation.
20      all s: UZHBusStation | lone station.s
21
22      // (v) There is at least one UZHBus.
23      some UZHBus
24  }
25
26  pred show {}
    Execute
27  run show for 2 but 1 UZHBusStation
```